SwarRakshak

> **When a voice can no longer be trusted by itself, SwarRakshak combines voice authenticity, speaker consistency, and call context to recommend the safest next action before fraud occurs.**

1. One-line description

SwarRakshak is a privacy-aware voice-risk assistant that analyzes call audio and request context to warn users about possible voice-cloning impersonation before they approve a sensitive action.

2. Problem

Artificial intelligence can generate voices that sound like real executives, customers, government officers, or family members. Attackers can use these cloned voices, together with leaked personal information, to make convincing calls.

A caller may ask an employee to approve a fund transfer, share a password, disclose confidential information, or bypass normal verification. Caller ID can be spoofed, and a familiar-sounding voice is no longer reliable proof of identity.

The problem is especially serious when employees are under pressure to act quickly. Existing defenses such as callbacks, multifactor authentication, fraud rules, and voice biometrics may exist, but users often do not receive a clear warning at the moment they are deciding whether to trust a call.

3. Solution

SwarRakshak analyzes three types of evidence:

1. **Voice authenticity:** Does the audio contain patterns associated with synthetic or manipulated speech?
2. **Speaker consistency:** Does the current voice resemble a trusted reference sample, when one is available?
3. **Call context:** Is the request urgent, unusual, coming from an unknown number, or connected to a high-value action?

The system combines these signals into an understandable risk level: **Low**, **Medium**, **High**, or **Inconclusive**. It explains the main reasons and recommends a safer next action, such as an

independent callback or second-person approval.

SwarRakshak is a decision-support tool. It does not prove that a caller is genuine or automatically approve or reject financial transactions.

4. Key features

- Upload a short current-call audio sample.
- Optionally upload a consented reference voice sample.
- Validate and preprocess audio files.
- Analyze audio for possible synthetic-speech patterns.
- Compare the current voice with a reference sample when available.
- Capture call context through a simple form.
- Calculate a transparent risk score from 0 to 100.
- Display Low, Medium, High, or Inconclusive risk.
- Explain the reasons contributing to the result.
- Show segment-level results for a simple analysis timeline.
- Recommend an appropriate verification action.
- Delete raw audio after analysis by default.
- Store metadata-only incident summaries.
- Provide REST endpoints for analysis and fraud-analyst review.

5. Demo

The hackathon demonstration uses three prepared scenarios.

Scenario 1: Genuine normal call

The voice matches the reference sample, the number is known, and the request follows normal procedure. SwarRakshak should produce a Low-risk result while reminding the user to follow normal verification procedures.

Scenario 2: Possible cloned voice

The audio contains patterns associated with synthetic speech, the voice differs from the reference sample, and the caller requests an urgent high-value transfer. SwarRakshak should

show a High-risk warning and recommend pausing the transfer and calling the person through an independently trusted number.

Scenario 3: Natural voice but suspicious request

The voice appears natural, but the caller uses an unknown number, creates urgency, and asks the employee to bypass a second approval. SwarRakshak should increase the risk level using contextual evidence.

Suggested demo flow

1. Introduce the fake executive-call problem.
2. Upload or select a prepared sample.
3. Enter the call context.
4. Run the analysis.
5. Show the score, reasons, and recommended action.
6. Repeat the flow for the other scenarios.
7. Explain privacy protections and model limitations.

6. Screenshots

Add screenshots of the implemented application here.

Main analysis screen

`<SCREENSHOT_PLACEHOLDER: analysis screen>`

High-risk result screen

`<SCREENSHOT_PLACEHOLDER: high-risk result>`

Fraud-analyst summary screen

`<SCREENSHOT_PLACEHOLDER: analysis summary>`

> Screenshot placeholders are intentional until the implemented React screens are captured.

7. Tech stack

The following stack is recommended for the hackathon implementation. Replace this section with the actual technologies used if the implementation differs.

Frontend

- React with simple CSS, or plain HTML, CSS, and JavaScript.
- Recharts or a simple CSS timeline for segment results.
- Browser microphone input may be added if time permits.

Backend

- Python.
- FastAPI or Flask.
- REST API under `/api/v1`.

Audio processing and machine learning

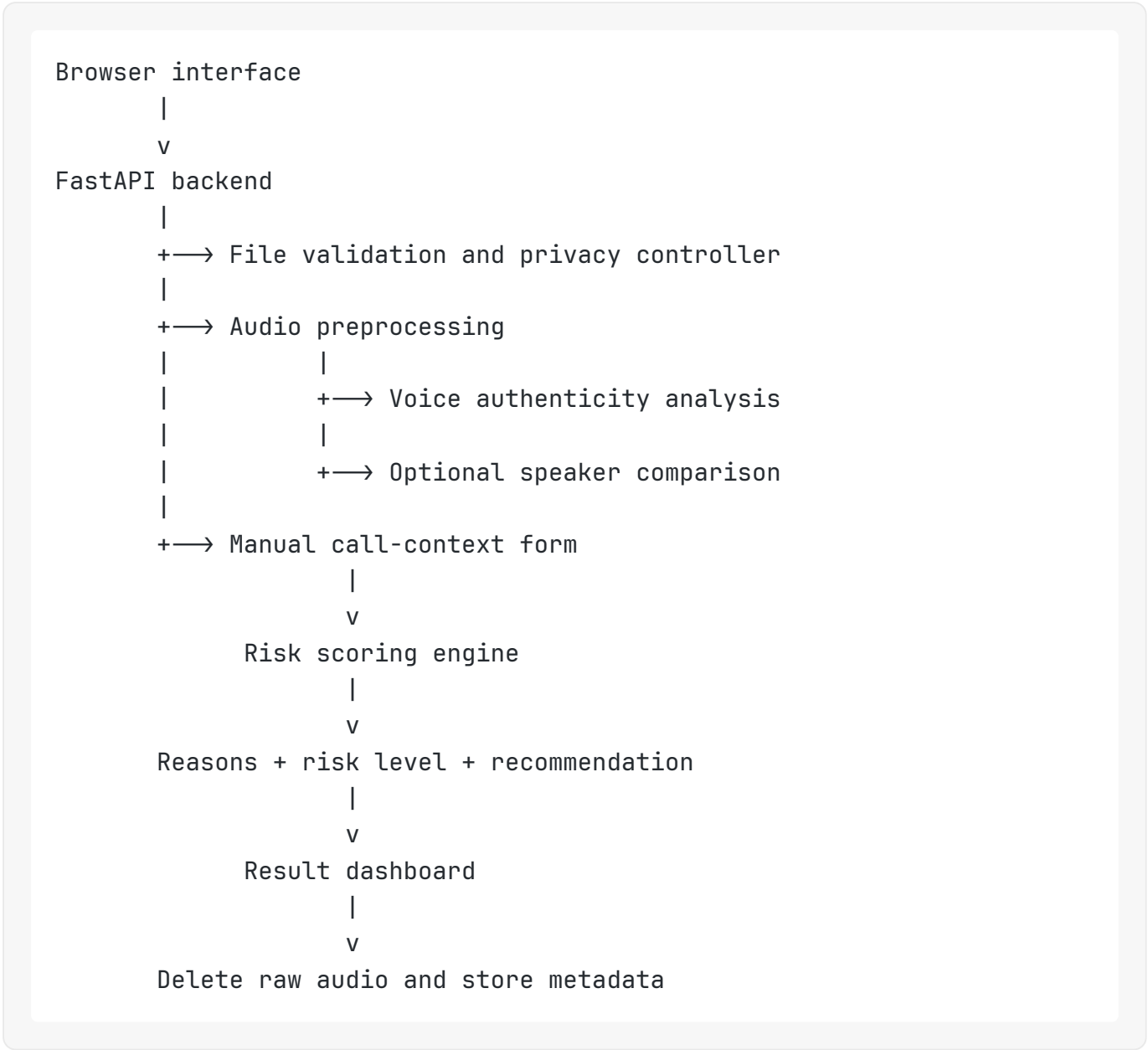
- FFmpeg for audio conversion.
- librosa for audio features and preprocessing.
- PyTorch or torchaudio for machine-learning audio pipelines.
- A suitable pretrained audio model or a clearly labeled prototype classifier.
- Optional speaker-embedding model for reference-voice comparison.
- Optional speech-to-text model such as Whisper.

Database and deployment

- SQLite for the hackathon database.
- Docker or a single hosted application for deployment.
- Temporary private storage for audio during processing.

8. Architecture overview

SwarRakshak uses a **modular monolith** for the hackathon. This means the application is deployed as one service, but its internal responsibilities are separated into modules.



Main modules

- **Web interface:** Uploads audio, collects context, and displays results.
- **API layer:** Coordinates requests and returns JSON responses.
- **Audio preprocessing:** Converts, validates, and segments audio.
- **Voice authenticity analysis:** Estimates whether audio may be synthetic or manipulated.
- **Speaker comparison:** Compares the current sample with a reference sample when available.
- **Context and policy module:** Converts call details into risk signals.
- **Risk scoring engine:** Combines signals into a score from 0 to 100.
- **Recommendation engine:** Suggests the safest next action.

- **Privacy and audit module:** Deletes raw audio and stores metadata-only results.

Privacy principle

Raw audio is not stored in the database for the MVP. It should be kept temporarily in a private location and deleted after analysis by default.

9. Project structure

The following is a suggested structure. Update it to match the actual repository.

```
swarrakshak/
├── frontend/                  # React or static web interface
│   ├── src/
│   ├── public/
│   └── package.json
├── backend/                   # FastAPI or Flask application
│   ├── app/
│   │   ├── api/               # REST routes
│   │   ├── audio/              # Validation and preprocessing
│   │   ├── models/             # Model adapters and inference
│   │   ├── scoring/            # Risk scoring and reason codes
│   │   ├── privacy/            # Temporary-file cleanup
│   │   ├── database/           # Database connection and repositories
│   │   └── main.py             # Application entry point
│   ├── requirements.txt
│   └── .env.example
├── database/
│   └── schema.sql              # SQLite schema
├── samples/                    # Consent-based or synthetic demo samples
├── docs/
│   └── screenshots/
├── README.md
└── docker-compose.yml          # Optional
```

> This structure is a suggested starting point. Do not add folders that are not used by the implementation.

10. Prerequisites

Install the following before running the project. Exact versions should be added by the team once the implementation is finalized.

- Python 3.11 or a compatible Python version.
- Node.js and npm or pnpm if the frontend uses React.
- FFmpeg for audio conversion.
- Git.
- A suitable Python package manager such as `pip`.
- Access to the selected audio-analysis model, if the project uses one.

Verify the main tools:

```
python --version
node --version
ffmpeg -version
git --version
```

11. Installation

Clone the repository

Replace the placeholder repository URL with the actual repository URL.

```
git clone <REPOSITORY_URL>
cd swarrakshak
```

Install backend dependencies

```
cd backend
python -m venv .venv
```

Activate the virtual environment.

macOS or Linux

```
source .venv/bin/activate
```

Windows PowerShell

```
.venv\Scripts\Activate.ps1
```

Install the dependencies:

```
pip install -r requirements.txt
```

Install frontend dependencies

If the frontend uses React:

```
cd ../frontend
npm install
```

If the project uses plain HTML, CSS, and JavaScript, skip this step.

12. Environment variables

Create a local environment file from the example file:

```
cp backend/.env.example backend/.env
```

The following names are suggested. Add only variables that the implementation actually uses.

```
# Backend
APP_ENV=development
API_HOST=127.0.0.1
API_PORT=8000

# Database
DATABASE_URL=sqlite:///./swarrakshak.db

# Audio processing
MAX_AUDIO_SIZE_MB=25
MAX_AUDIO_DURATION_SECONDS=60
TEMP_AUDIO_DIR=./tmp/audio
RETAIN_AUDIO_BY_DEFAULT=false

# Model configuration
AUDIO_MODEL_NAME=<MODEL_NAME_IF_USED>
SPEAKER_MODEL_NAME=<MODEL_NAME_IF_USED>
MODEL_VERSION=prototype-audio-1
RULES_VERSION=risk-rules-1

# Optional authentication
AUTH_ENABLED=false
```

Do not commit `.env` files containing credentials or private configuration. Keep `.env.example` free of secrets.

13. How to run the frontend

The exact command depends on the frontend implementation.

React example

```
cd frontend
npm run dev
```

The development server usually prints the local URL in the terminal. Use the URL shown by the command.

Static frontend example

```
cd frontend
python -m http.server 5173
```

Open:

```
http://localhost:5173
```

If the frontend uses a different command or port, update this section.

14. How to run the backend

From the backend directory, activate the virtual environment and run the API.

FastAPI example

```
cd backend
source .venv/bin/activate
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

The API should then be available at:

```
http://localhost:8000
```

Interactive API documentation, when enabled by FastAPI, is available at:

```
http://localhost:8000/docs
```

Check the health endpoint

```
curl http://localhost:8000/api/v1/health
```

Expected response shape:

```
{
  "status": "ok",
  "service": "swarrakshak-api",
  "version": "0.1.0",
  "audio_model": "ready",
  "speaker_model": "ready"
}
```

15. Database setup

SwarRakshak uses SQLite for the hackathon MVP.

Create the database schema

From the project root:

```
sqlite3 swarrakshak.db < database/schema.sql
```

If the project initializes the database from Python, use the project's database initialization command instead.

MVP tables

- `users` — optional demo and application users.
- `analysis_events` — one record for each analysis attempt.
- `analysis_context` — caller and transaction context.
- `analysis_results` — final score, risk level, confidence, and recommendation.

- `analysis_reasons` — explanations and score contributions.
- `analysis_segments` — segment-level analysis results.

Database privacy rule

Do not store raw audio bytes in SQLite. Store only the analysis metadata needed for the result and audit summary.

16. API documentation

The MVP exposes four REST endpoints:

Method	Endpoint	Purpose
`GET`	`/api/v1/health`	Check API and model availability
`POST`	`/api/v1/analyses`	Create and run a call analysis
`GET`	`/api/v1/analyses/{event_id}`	Get one analysis result
`GET`	`/api/v1/analyses`	List metadata-only analysis summaries

Create an analysis

```
POST /api/v1/analyses
Content-Type: multipart/form-data
```

Example:

```
curl -X POST http://localhost:8000/api/v1/analyses \
  -F "current_audio=@samples/current_call.wav" \
  -F "reference_audio=@samples/reference_voice.wav" \
  -F "caller_number_status=unknown" \
  -F "request_urgency=urgent" \
  -F "transaction_value=high" \
  -F "procedure_status=bypass_requested" \
  -F "caller_relationship=executive" \
  -F "retain_audio=false"
```

Example response:

```
{
  "event_id": "evt_01J9SWR123",
  "status": "completed",
  "risk_score": 82,
  "risk_level": "high",
  "confidence_level": "high",
  "confidence_score": 0.78,
  "evidence_sufficient": true,
  "reasons": [
    "Audio contains patterns associated with synthetic speech.",
    "Current voice differs from the reference sample.",
    "Caller number is not recognized as trusted."
  ],
  "recommended_action": "Pause the transfer and call the person using an
  "raw_audio_retained": false
}
```

Get one analysis

```
GET /api/v1/analyses/{event_id}
```

Example:

```
curl http://localhost:8000/api/v1/analyses/evt_01J9SWR123
```

List analysis summaries

```
GET /api/v1/analyses?risk_level=high&page=1&page_size=20
```

This endpoint is intended for fraud analysts and administrators. It returns metadata and explanations, not raw audio or full transcripts.

Common status codes

Status	Meaning
`200`	Request completed successfully
`202`	Request accepted for asynchronous processing, if implemented
`400`	Invalid request or query parameter
`401`	Authentication required or invalid
`403`	User lacks permission
`404`	Analysis event not found
`413`	Audio file is too large
`415`	Audio format is unsupported
`422`	Audio cannot be decoded or analyzed
`500`	Unexpected server error
`503`	Analysis model or service unavailable

For the complete field-level API contract, see `swarrakshak_api_specification.md`.

17. Testing

Run backend tests

Replace the command if the project uses a different test runner.

```
cd backend
pytest
```

Test the health endpoint

```
curl http://localhost:8000/api/v1/health
```

Test the three demo scenarios

The project should test:

1. Genuine voice with normal context.
2. Possible synthetic or cloned voice with suspicious context.
3. Natural-looking voice with a suspicious high-value request.

Test important edge cases

- Missing current audio.
- Unsupported file type.
- Audio file that is too large.
- Audio that is too short or silent.
- No reference sample provided.
- Unknown caller number.
- Urgent request.
- High-value transaction.
- Request to bypass normal procedure.
- Model unavailable.
- Temporary audio deletion after analysis.
- Unauthorized access to another user’s analysis.

Test privacy behavior

Confirm that:

- Raw audio is not stored in the database.
- Temporary audio is deleted after analysis by default.
- API responses do not contain raw audio.

- Logs do not contain access tokens or sensitive audio data.

18. Deployment

Hackathon deployment

The simplest deployment is a single application containing:

- Frontend.
- FastAPI backend.
- SQLite metadata database.
- Local or hosted analysis model.
- Private temporary audio directory.

Before deployment:

1. Set `AUTH_ENABLED` according to the actual implementation.
2. Configure allowed frontend origins.
3. Use HTTPS when the application is accessed over a network.
4. Confirm that the temporary audio directory is not publicly accessible.
5. Confirm that raw audio is deleted after analysis.
6. Use only synthetic or consented demo samples.

Docker placeholder

If Docker is implemented, add the actual command here:

```
docker compose up --build
```

Current status: Docker configuration has not been provided in the project information. Add this command only after the team creates and tests the relevant files.

Deployed URL

```
<DEPLOYED_APPLICATION_URL>
```

Replace the placeholder after deployment.

19. Future improvements

These features are possible future improvements and are not required for the first hackathon MVP:

- Browser microphone input with live segment updates.
- Support for Hindi and additional Indian languages.
- Accent-aware evaluation.
- Speech-to-text and suspicious-intent detection.
- Detection of replayed recordings.
- Integration with enterprise contact directories.
- SMS, email, or in-app alerts.
- Configurable rules for different departments and transaction limits.
- Investigator dashboards and feedback workflows.
- On-device or edge inference.
- Scalable inference workers and model monitoring.
- Enterprise single sign-on and role-based access.
- Integration with telecom, VoIP, and collaboration platforms.

The team should not add these features until the basic analysis, explanation, recommendation, and privacy workflow is reliable.

20. Limitations and responsible use

SwarRakshak is a prototype and its result is probabilistic. It may produce false positives or false negatives. Audio quality, network compression, microphones, illness, language, and accent can affect performance.

A Low-risk result does not prove that a caller is genuine. A High-risk result means that the user should slow down and verify independently.

SwarRakshak must not automatically:

- Move or approve money.
- Reject a customer or employee.
- Make legal, medical, employment, government, or financial decisions.
- Block or terminate calls.

Use synthetic or consented voice samples for demonstrations. Do not use real confidential calls or financial credentials.

21. Team members

Add the actual team members here.

Name	Role	Contribution
`<TEAM_MEMBER_1>`	`<ROLE>`	`<CONTRIBUTION>`
`<TEAM_MEMBER_2>`	`<ROLE>`	`<CONTRIBUTION>`
`<TEAM_MEMBER_3>`	`<ROLE>`	`<CONTRIBUTION>`
`<TEAM_MEMBER_4>`	`<ROLE>`	`<CONTRIBUTION>`

Related project documents

- Product Requirements Document
- System Architecture
- Database Design
- SQLite Schema
- REST API Specification

License

`<ADD_LICENSE_INFORMATION_HERE>`

No license information was provided. Add a license before publishing the repository.